torch.linalg.lu_factor#

https://pytorch.org/docs/stable/generated/torch.linalg.lu_factor.html

Description:

Computes a compact representation of the LU factorization with partial pivoting of a matrix. This function computes a compact representation of the decomposition given by torch.linalg.lu(). If the matrix is square, this representation may be used in torch.linalg.lu_solve() to solve system of linear equations that share the matrix A. The returned decomposition is represented as a named tuple (LU, pivots). The LU matrix has the same shape as the input matrix A. Its upper and lower triangular parts encode the non-constant elements of L and U of the LU decomposition of A. The returned permutation matrix is represented by a 1-indexed vector. pivots[i] == j represents that in the i-th step of the algorithm, the i-th row was permuted with the j-1-th row. On CUDA, one may use pivot= False. In this case, this function returns the LU decomposition without pivoting if it exists.

Function Signature

```
torch.linalg.lu_factor(A, *, bool pivot=True, out=None) ->
(Tensor, Tensor)#

Parameters

Keyword Arguments

Returns

Raises
```

Parameters

Keyword

pivot (bool, optional) – Whether to compute the LU decomposition with partial pivoting, or the regular LU decomposition. pivot= False not supported on CPU. Default: True. out (tuple, optional) – tuple of two tensors to write the output to. Ignored if None. Default: None.

Returns:

A named tuple (LU, pivots).

Code Examples

```
>>> A = torch.randn(2, 3, 3)
>>> B1 = torch.randn(2, 3, 4)
>>> B2 = torch.randn(2, 3, 7)
>>> LU, pivots = torch.linalg.lu_factor(A)
>>> X1 = torch.linalg.lu_solve(LU, pivots, B1)
>>> X2 = torch.linalg.lu_solve(LU, pivots, B2)
>>> torch.allclose(A @ X1, B1)
True
>>> torch.allclose(A @ X2, B2)
True
```

Notes & Warnings

NOTE

Note When inputs are on a CUDA device, this function synchronizes that device with the CPU. For a version of this function that does not synchronize, see torch.linalg.lu_factor_ex().

⚠ WARNING Warning The LU decomposition is almost never unique, as often there are different permutation matrices that can yield different LU decompositions. As such, different platforms, like SciPy, or inputs on different devices, may produce different valid decompositions. Gradient computations are only supported if the input matrix is full-rank. If this condition is not met, no error will be thrown, but the gradient may not be finite. This is because the LU decomposition with pivoting is not differentiable at these points.

NOTE See also torch.linalg.lu_solve() solves a system of linear equations given the output of this function provided the input matrix was square and invertible. torch.lu_unpack() unpacks the tensors returned by lu_factor() into the three matrices P, L, U that form the decomposition. torch.linalg.lu() computes the LU decomposition with partial pivoting of a possibly non-square matrix. It is a composition of lu_factor() and torch.lu_unpack(). torch.linalg.solve() solves a system of linear equations. It is a composition of lu_factor() and lu_solve().

Detailed Documentation

Documentation

Computes a compact representation of the LU factorization with partial pivoting of a matrix.

This function computes a compact representation of the decomposition given by torch.linalg.lu(). If the matrix is square, this representation may be used in

torch.linalg.lu_solve() to solve system of linear equations that share the matrix A.

The returned decomposition is represented as a named tuple (LU, pivots). The LU matrix has the same shape as the input matrix A. Its upper and lower triangular parts encode the non-constant elements of L and U of the LU decomposition of A.

The returned permutation matrix is represented by a 1-indexed vector. pivots[i] == j represents that in the i-th step of the algorithm, the i-th row was permuted with the j-1-th row.

On CUDA, one may use pivot= False. In this case, this function returns the LU decomposition without pivoting if it exists.

Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

When inputs are on a CUDA device, this function synchronizes that device with the CPU. For a version of this function that does not synchronize, see torch.linalg.lu_factor_ex().

The LU decomposition is almost never unique, as often there are different permutation matrices that can yield different LU decompositions. As such, different platforms, like SciPy, or inputs on different devices, may produce different valid decompositions.

Gradient computations are only supported if the input matrix is full-rank. If this condition is not met, no error will be thrown, but the gradient may not be finite. This is because the LU decomposition with pivoting is not differentiable at these points.

torch.linalg.lu_solve() solves a system of linear equations given the output of this function provided the input matrix was square and invertible.

torch.lu_unpack() unpacks the tensors returned by lu_factor() into the three matrices P, L, U that form the decomposition.

torch.linalg.lu() computes the LU decomposition with partial pivoting of a possibly non-square matrix. It is a composition of lu_factor() and torch.lu_unpack().

torch.linalg.solve() solves a system of linear equations. It is a composition of lu_factor() and lu_solve().

A (Tensor) – tensor of shape (*, m, n) where * is zero or more batch dimensions.

pivot (bool, optional) – Whether to compute the LU decomposition with partial pivoting, or the regular LU decomposition. pivot= False not supported on CPU. Default: True.

out (tuple, optional) – tuple of two tensors to write the output to. Ignored if None. Default: None.

A named tuple (LU, pivots).

RuntimeError – if the A matrix is not invertible or any matrix in a batched A is not invertible.